

# Python Cache Cheatsheet

## Master cachetools for Faster & Smarter Python Apps

**Cache it once. Use it many times. Save time & resources!**

Note: Only my left arm is visible in perspective.

### How Caching Works




## 1 LRU Cache (Least Recently Used)

Removes the least recently used item when the cache is full.

```python
from cachetools import LRUCache, cached
cache = LRUCache(maxsize=3)

@cached(cache)
def compute(x):
    print(f"Computing {x}...")
    return x * x
```

**LRU Flow (maxsize=3)**




Ideal for scenarios where recent data is more likely to be requested again.

## 2 TTL Cache (Time To Live)

Items expire after a set time (in seconds).

```python
from cachetools import TTLCache, cached
import time
mi_cache = TTLCache(maxsize=50, ttl=1)  # 1 sec

@cached(mi_cache)
def consulta_pesada(id_producto: int):
    print(f"--- Consultando DB para {id_producto) ---")
    time.sleep(2)  # Simulating slowness
    return f"Datos del producto (id_producto)"
```




Perfect for temporary data like API responses, sessions, tokens, or volatile data.

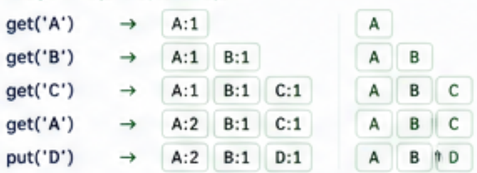
## 3 LFU Cache (Least Frequently Used)

Removes the least frequently used item.

```python
from cachetools import LFUCache, cached
lfu_cache = LFUCache(maxsize=3)

@cached(lfu_cache)
def get_data_lfu(key):
    print(f"Fetching data for key (key) (LFU)...")
    return f"Data for {key)"
```

**Usage Frequency Example**




Great when popular data should stay in cache longer.

## 4 Custom Cache Key Generation

Create custom keys when default args or a subset should form the cache key.

```python
from cachetools import LRUCache, cached
import json
custom_key_cache = LRUCache(maxsize=2)

def custom_key_generator(list_arg, dict_arg, simple_arg):
    sorted_dict_str = json.dumps(dict_arg, sort_keys=True)
    list_tuple = tuple(list_arg)
    return (list_tuple, sorted_dict_str, simple_arg)

@cached(custom_key_cache, key=custom_key_generator)
def process_complex_data(data_list, settings_dict, op_type):
    # Heavy computation....
    return f"Result for {op_type)"
```

Useful when using lists, dicts or needing control over cache keys.

## 5 Manual Cache Invalidation

Invalidate specific entries or clear the entire cache.

```python
from cachetools import LRUCache, cached
manual_invalidation_cache = LRUCache(maxsize=5)

@cached(manual_invalidation_cache)
def get_user_profile(user_id):
    print(f"Fetching user {user_id} from DB...")
    return {'id': user_id, 'name': f'User (user_id)', 'status': 'active'}
```

**1. Invalidate a Specific Entry**

```python
del manual_invalidation_cache((1,))
# Keys are tuples of arguments
```




**2. Clear the Entire Cache**

```python
manual_invalidation_cache.clear()
```




Ideal after data updates, config changes or system events.

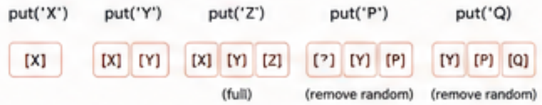
## 6 RR Cache (Random Replacement)

Randomly removes an item when the cache is full.

```python
from cachetools import RRCache, cached
rr_cache = RRCache(maxsize=3)

@cached(rr_cache)
def get_data_rr(key):
    print(f"Fetching data for key (key) (RR)...")
    return f"Data for {key)"
```

**Example Flow (maxsize=3)**




Good for simple, non-critical caches where exact eviction order doesn't matter.

## 7 FIFO Cache (First-In First-Out)

Removes the oldest item in the cache first.

```python
from cachetools import FIFOCache, cached
fifo_cache = FIFOCache(maxsize=3)

@cached(fifo_cache)
def get_data_fifo(key):
    print(f"Fetching data for key (key) (FIFO)...")
    return f"Data for {key)"
```

**FIFO Flow (maxsize=3)**




Works like a queue: oldest goes out first, regardless of usage.

## 8 Cache Comparison Overview

| Strategy | Eviction Criteria | Ideal For |
|---|---|---|
| LRU | Least recently used | General use, recent access matters |
| TTL | Time-based expiration | Temporary data, APIs, sessions |
| LFU | Least frequently used | Popular & reused data |
| RR | Random | Simple cases or low precision needs |
| FIFO | Oldest first | Queues, batches, streams |

Choose based on your access pattern and the nature of your data.

## Best Practices

**Choose the Right Size**
Set maxsize according to available memory.

**Monitor & Tune**
Review hit rate, size and adjust accordingly.

**Invalidate Carefully**
Don't forget to invalidate or clear caches when data changes.

**Avoid Over-Caching**
Not everything needs caching. Evaluate the cost vs benefit.

**Use Clear Keys**
Simple, consistent keys avoid bugs and duplicates.

**Supported Python Versions**
Works with Python 3.8+ and above.

```
pip install cachetools
```

**About cachetools**
A powerful & flexible in-memory caching library for Python.



Cache smart. Compute less. Deliver more.

Happy Caching!